Project 1 Report: Heart Disease Prediction from the BRFSS Survey

Sacha Luca Magnollay*, Thibaud Jack Pier Babin†, Haris Maxime Aouini‡
*SCIPER 378865, †SCIPER 378845, ‡SCIPER 373506
École Polytechnique Fédérale de Lausanne (EPFL)
Machine Learning CS-433, Autumn 2026

Summary

We had to predict whether a person has coronary heart disease from their answers to the 2015 BRFSS health survey. We first implemented the six required linear methods in NumPy, cleaned the survey answers and chose the decision threshold that maximizes the F1 score. The best linear model, regularized logistic regression, reached a cross-validated F1 of 0.4290. We then implemented gradient-boosted decision trees, which reached 0.4357 and are used for our final submission.

I. Task and Data

Each sample is one survey respondent described by 321 answers, and the label (MICHD) says whether they were diagnosed with coronary heart disease [1]. The training set has 328,135 samples and the test set 109,379. Only 8.8% of the training labels are positive, so predicting "healthy" for everyone already gives 91% accuracy. For this reason we focus on the F1 score of the positive class. All results below come from stratified 5-fold cross-validation, with the same folds for every model.

II. Preprocessing (Linear Models)

We applied the following steps, with every statistic computed on the training set only:

1) drop survey metadata (dates, weights, sequence numbers) and columns with more than 80% missing values;
2) decode the special BRFSS codes: 88, 888 and 555 mean "none" ($\rightarrow$ 0), 77, 99, 777 and 999 mean "don't know" or "refused" ($\rightarrow$ missing);
3) one-hot encode columns with at most 15 distinct values, with missing as its own category;
4) for the other columns, replace missing values by the median, add a missing indicator and clip to the 0.1%–99.9% quantiles;
5) remove columns that are linear combinations of others, standardize and add a bias column.

This gives 371 features. We also tried a smaller set of 21 hand-picked risk factors [2], [3].

III. Models

Linear models. We implemented least squares and ridge regression (normal equations), linear regression with gradient descent and SGD, and logistic regression with and without ℓ_2 penalty. We tuned the hyperparameters on a small grid. The best one is regularized logistic regression ($\lambda = 10^{-4}$, $\gamma = 0.5$, 500 iterations).

Gradient-boosted trees. Linear models cannot combine features (e.g. "old and diabetic"), and trees usually work better on tabular data [4]. We therefore wrote our own gradient boosting in NumPy, in the style of XGBoost [5]. Each raw column is cut into 64 bins and missing values get their own bin, so this model needs none of the preprocessing above. We add trees of depth 3 one by one (learning rate 0.1), each trained on a random half of the rows and columns, and stop when the loss on a held-out 10% of the training set stops improving. We removed the column HAREHAB1 (cardiac rehabilitation): it is only asked to people who had a heart attack, so it gives away the label.

Decision threshold. With so few positives, the default threshold (probability 0.5) predicts almost nobody as sick. For each model we choose the threshold that gives the best F1 on the cross-validation predictions.

IV. Results

Table I shows the results. The boosted trees get the best F1, 0.4357 ($\pm$0.0028 between folds), against 0.4290 for regularized logistic regression. The features they use the most are general health, stroke, age, sex and diabetes. The linear models are all between 0.419 and 0.429, except SGD, which converges slowly.

The two choices that helped the most were using all features instead of the 21 selected ones (0.4125 $\rightarrow$ 0.4290) and choosing the threshold (Figure 1): with the default threshold, F1 is only 0.233.

V. What Did Not Work

We tried several ways to improve the trees, but none changed F1 by more than the noise between folds ($\pm$0.003): extra hand-made features (risk-factor counts, target encoding), a smaller learning rate, more bins, other models (additive models, random forests) and averaging several models (best 0.4373). All these

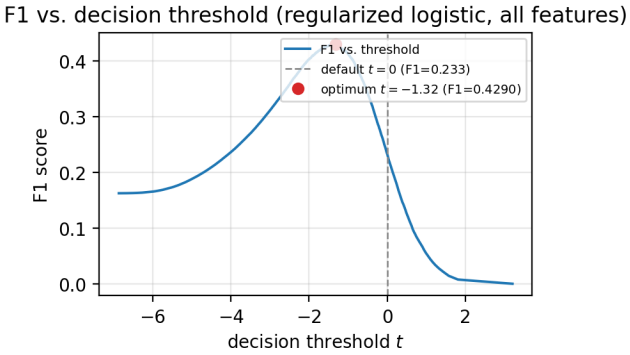


Figure 1. F1 as a function of the decision threshold t for regularized logistic regression. Red dot: best threshold ($t \approx -1.315$, F1 = 0.4290); dashed line: default $t = 0$ (F1 = 0.233).

Table I
5-fold cross-validation results on all features, each with its best threshold. Ridge: λ=10^{-3}; logistic: γ=0.5, 500 iterations; reg. logistic: λ=10^{-4}.

Model	F1	Accuracy	Threshold
Least squares	0.4197	0.8677	0.219
Ridge	0.4199	0.8644	0.214
Linear reg. (GD)	0.4187	0.8730	0.225
Linear reg. (SGD)	0.3168	0.8252	0.254
Logistic reg.	0.4288	0.8736	−1.361
Reg. logistic	0.4290	0.8768	−1.315
Boosted trees	0.4357	0.8722	−1.24

models make the same mistakes: their predictions are 95–99% correlated. Keeping HAREHAB1 would have given 0.444, but only by leaking the label.

VI. Conclusion

Our final model, gradient-boosted trees with a tuned threshold, reaches a cross-validated F1 of 0.4357. As often in practice [6], using all the data and choosing the threshold mattered more than the choice of model. Since many different attempts all stay around 0.436, we think this is close to what can be predicted from these survey answers. python run.py reproduces our submission and python run_linear.py the linear results.

References

[1] Centers for Disease Control and Prevention (CDC), "Behavioral risk factor surveillance system survey data, 2015," https://www.cdc.gov/brfss/annual_data/annual_2015.html, 2015, accessed: 2026-10-27.

[2] Z. Xie, O. Nikolayeva, J. Luo, and D. Li, "Building risk prediction models for type 2 diabetes using machine learning techniques," Preventing Chronic Disease, vol. 16, p. 190109, 2019.

[3] A. Teboul, "Heart disease health indicators dataset notebook," https://www.kaggle.com/code/alexteboul/heart-disease-health-indicators-dataset-notebook, kaggle Notebook, accessed 2026-10-27.

[4] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in NeurIPS Datasets and Benchmarks, 2022.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. ACM SIGKDD, 2016, pp. 785–794.

[6] P. Domingos, "A few useful things to know about machine learning," Communications of the ACM, vol. 55, no. 10, pp. 78–87, 2012.